

Lab1

Course: ARTI 404 – Image Processing

Lab Topic: NumPy Operations and Image Loading & Visualization

Student Name: Muhannad Almutlaq

Student ID: 2240006060

1. Lab Outcomes

The objective of this lab is to practice creating and manipulating NumPy arrays, and to load, visualize, convert, and save images as NumPy arrays using OpenCV, PIL, and Matplotlib.

2. Procedural Tasks

Task 1: NumPy Operations

- **Description:** Imported the NumPy library and worked through its core array features: creation, input/output, data types, mathematical and statistical operations, indexing, and structural manipulation.

Task 1.1: Creating Arrays of Different Shapes and Data Types

- **Description:** Created 1-D, 2-D, and 3-D arrays from Python lists and tuples, using the dtype parameter to build float and integer versions, and printed each array with its shape.
- **Code & Output:**

```
import numpy as np
x= np.array([1, 2, 3])
y1= np.array([(1.5, 2, 3), (4, 5, 6)], dtype= float)
y2= np.array([(1.5, 2, 3), (4, 5, 6)], dtype= int)
z1= np.array([(1.5, 2, 3), (4, 5, 6)], [(3, 2, 1), (4, 5, 6)]], dtype= float)
z2= np.array([(1.5, 2, 3), (4, 5, 6)], [(3, 2, 1), (4, 5, 6)]], dtype= int)

print("They should have similar shapes. Otherwise, it will raise an error.")
print(f"X: {x}\nShape: {x.shape}\n")
print(f"Y1: {y1}\nShape: {y1.shape}\n")
print(f"Y2: {y2}\nShape: {y2.shape}\n")
print(f"Z1: {z1}\nShape: {z1.shape}\n")
```

```

print(f"Z2: {z2}\nShape: {z2.shape}\n")
They should have similar shapes. Otherwise, it will raise an error.
X: [1 2 3]
Shape: (3,)

Y1: [[1.5 2.  3. ]
      [4.  5.  6. ]]
Shape: (2, 3)

Y2: [[1 2 3]
      [4 5 6]]
Shape: (2, 3)

Z1: [[[1.5 2.  3. ]
      [4.  5.  6. ]]
      [[3.  2.  1. ]
      [4.  5.  6. ]]]
Shape: (2, 2, 3)

Z2: [[[1 2 3]
      [4 5 6]]
      [[3 2 1]
      [4 5 6]]]
Shape: (2, 2, 3)

```

Task 1.2: Array Generation Functions

- **Description:** Generated arrays with `np.zeros`, `np.ones`, `np.arange`, `np.linspace`, `np.full`, `np.eye`, `np.random.random`, and `np.empty` using different shapes, ranges, and data types. `np.empty` does not initialize its memory, so its values are arbitrary.
- **Code & Output:**

```

np.zeros((2, 4))
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.]])

np.zeros((2, 2, 4))
array([[[0., 0., 0., 0.],
        [0., 0., 0., 0.]],
       [[0., 0., 0., 0.],
        [0., 0., 0., 0.]])

```

```

np.ones((2, 4), dtype= np.int16)
array([[1, 1, 1, 1],
       [1, 1, 1, 1]], dtype=int16)

np.ones((2, 2, 4), dtype= np.int16)
array([[[1, 1, 1, 1],
        [1, 1, 1, 1]],

       [[1, 1, 1, 1],
        [1, 1, 1, 1]]], dtype=int16)

d1= np.arange(10, 25, 2)
d1
array([10, 12, 14, 16, 18, 20, 22, 24])

d2= np.arange(10, 25)
d2
array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24])

np.linspace(0, 2, 9)
array([0. , 0.25, 0.5 , 0.75, 1. , 1.25, 1.5 , 1.75, 2. ])

np.linspace(0, 2)
array([0. , 0.04081633, 0.08163265, 0.12244898, 0.16326531,
        0.20408163, 0.24489796, 0.28571429, 0.32653061, 0.36734694,
        0.40816327, 0.44897959, 0.48979592, 0.53061224, 0.57142857,
        0.6122449 , 0.65306122, 0.69387755, 0.73469388, 0.7755102 ,
        0.81632653, 0.85714286, 0.89795918, 0.93877551, 0.97959184,
        1.02040816, 1.06122449, 1.10204082, 1.14285714, 1.18367347,
        1.2244898 , 1.26530612, 1.30612245, 1.34693878, 1.3877551 ,
        1.42857143, 1.46938776, 1.51020408, 1.55102041, 1.59183673,
        1.63265306, 1.67346939, 1.71428571, 1.75510204, 1.79591837,
        1.83673469, 1.87755102, 1.91836735, 1.95918367, 2. ])

np.linspace(0, 4, 10)
array([0. , 0.44444444, 0.88888889, 1.33333333, 1.77777778,
        2.22222222, 2.66666667, 3.11111111, 3.55555556, 4. ])

np.linspace(0, 1, 4)
array([0. , 0.33333333, 0.66666667, 1. ])

e1= np.full((2, 2), 7)
e1
array([[7, 7],
       [7, 7]])

e2= np.full((2, 4), 1)

```

```

e2
array([[1, 1, 1, 1],
       [1, 1, 1, 1]])

h1= np.eye(2)
h1
array([[1., 0.],
       [0., 1.]])

h1= np.eye(4)
h1
array([[1., 0., 0., 0.],
       [0., 1., 0., 0.],
       [0., 0., 1., 0.],
       [0., 0., 0., 1.]])

np.random.random((2, 2))
array([[0.63607596, 0.02312378],
       [0.11789419, 0.44434549]])

np.empty((2, 3))
array([[1.5, 2. , 3. ],
       [4. , 5. , 6. ]])

np.empty((3, 2), dtype= np.int16)
array([[257, 257],
       [257, 257],
       [257, 257]], dtype=int16)

np.empty((3, 2))
array([[1.5, 2. ],
       [3. , 4. ],
       [5. , 6. ]])

np.empty((3, 2), dtype= np.float16)
array([[0., 0.],
       [0., 0.],
       [0., 0.]], dtype=float16)

```

Task 1.3: Saving and Loading Arrays

- **Description:** Saved and loaded arrays in the .npy, .npz, and text formats with np.save, np.savez, np.load, np.savetxt, np.loadtxt, and np.genfromtxt, trying different delimiters. The two text files read with loadtxt and genfromtxt contained no data, so both calls returned empty arrays (with a warning).

- **Code & Output:**

```
np.save("r1", x)
np.savez("r2.npz", y1, y2)
np.load("r1.npy")
array([1, 2, 3])

np.loadtxt("txt1.txt")
array([], dtype=float64)

np.genfromtxt("file.csv", delimiter= ",", ")
array([], dtype=float64)

np.savetxt("r1.txt", x, delimiter= " ")
np.savetxt("r1v1.txt", x)
np.savetxt("r1v2.txt", x, delimiter= ",", ")
np.savetxt("r1v3.txt", x, delimiter= ".")
np.savetxt("r1v4.txt", x, newline= " ")
```

Task 1.4: NumPy Data Types

- **Description:** Printed the NumPy scalar types for integers, floating-point numbers, complex numbers, booleans, objects, and strings.
- **Code & Output:**

```
print(f"int64: {np.int64}")
int64: <class 'numpy.int64'>

print(f"float32: {np.float32}")
float32: <class 'numpy.float32'>

print(f"complex: {np.complex_}")
complex: <class 'numpy.complex128'>

print(f"bool: {np.bool_}")
bool: <class 'numpy.bool_'>

print(f"object: {np.object_}")
object: <class 'numpy.object_'>

print(f"string_: {np.string_}")
string_: <class 'numpy.bytes_'>

print(f"unicode_: {np.unicode_}")
unicode_: <class 'numpy.str_'>
```

Task 1.5: Arithmetic Operations, Mathematical Functions, and Dot Products

- **Description:** Performed element-wise subtraction, addition, division, and multiplication with both operators and the np.subtract, np.add, np.divide, and np.multiply functions. Applied exp, sqrt, sin, cos, and log, and computed dot products of 1-D and 2-D arrays with compatible shapes.
- **Code & Output:**

```
p= x - y1
p
array([[ -0.5,  0. ,  0. ],
       [-3. , -3. , -3. ]])

np.subtract(x, y1)
array([[ -0.5,  0. ,  0. ],
       [-3. , -3. , -3. ]])

x + y1
array([[2.5, 4. , 6. ],
       [5. , 7. , 9. ]])

np.add(x, y1)
array([[2.5, 4. , 6. ],
       [5. , 7. , 9. ]])

x / y1
array([[0.66666667, 1. , 1. ],
       [0.25 , 0.4 , 0.5 ]])

np.divide(x, y1)
array([[0.66666667, 1. , 1. ],
       [0.25 , 0.4 , 0.5 ]])

x * y1
array([[ 1.5,  4. ,  9. ],
       [ 4. , 10. , 18. ]])

np.multiply(x, y1)
array([[ 1.5,  4. ,  9. ],
       [ 4. , 10. , 18. ]])

np.exp(x)
array([ 2.71828183,  7.3890561 , 20.08553692])
```

```

np.sqrt(x)
array([1.          , 1.41421356, 1.73205081])

np.sin(x)
array([0.84147098, 0.90929743, 0.14112001])

np.cos(x)
array([ 0.54030231, -0.41614684, -0.9899925 ])

np.log(x)
array([0.          , 0.69314718, 1.09861229])

dot1= np.array([1, 2, 3, 4])
dot2= np.array([2, 4, 6, 8])

print(f"dot1: {dot1}\nShape: {dot1.shape}\n")
print(f"dot2: {dot2}\nShape: {dot2.shape}\n")
dot1: [1 2 3 4]
Shape: (4,)

dot2: [2 4 6 8]
Shape: (4,)

dot1.dot(dot2)
60

dot3= np.array([[1, 2, 3, 4]])
dot4= np.array([(2, 4, 6, 8), (10, 12, 14, 16), (18, 20, 22, 24), (26, 28, 30, 32)])

print(f"dot3: {dot3}\nShape: {dot3.shape}\n")
print(f"dot4: {dot4}\nShape: {dot4.shape}\n")
dot3: [[1 2 3 4]]
Shape: (1, 4)

dot4: [[ 2  4  6  8]
 [10 12 14 16]
 [18 20 22 24]
 [26 28 30 32]]
Shape: (4, 4)

dot3.dot(dot4)
array([[180, 200, 220, 240]])

dot5= np.array([[1, 2, 3, 4]])
dot6= np.array([(2, 4), (6, 8), (10, 12), (14, 16)])

```

```

print(f"dot5: {dot5}\nShape: {dot5.shape}\n")
print(f"dot6: {dot6}\nShape: {dot6.shape}\n")
dot5: [[1 2 3 4]]
Shape: (1, 4)

dot6: [[ 2  4]
 [ 6  8]
 [10 12]
 [14 16]]
Shape: (4, 2)

dot5.dot(dot6)
array([[100, 120]])

```

Task 1.6: Comparison Operations

- **Description:** Compared arrays element-wise with `==`, `<`, and `>`, compared an array against a scalar, `None`, and Python types, and tested whole-array equality with `np.array_equal`.
- **Code & Output:**

```

x == y1
array([[False,  True,  True],
       [False, False, False]])

y1 == y2
array([[False,  True,  True],
       [ True,  True,  True]])

x < 2
array([ True, False, False])

x > 2
array([False, False,  True])

x == 1
array([ True, False, False])

x == None
array([False, False, False])

x == int
array([False, False, False])

```

```

x == float
array([False, False, False])

np.array_equal(x, y1)
False

np.array_equal(y1, y2)
False

np.array_equal(y1, y1)
True

np.array_equal(y2, y2)
True

```

Task 1.7: Aggregate Functions and Statistics

- **Description:** Calculated the sum, minimum, maximum (over the whole array and along an axis), cumulative sum, mean, standard deviation, median, and correlation coefficient of arrays.
- **Code & Output:**

```

x.sum()
6

x.min()
1

x.max()
3

y1.min()
1.5

y1.min(axis= 1)
array([1.5, 4. ])

y1.max()
6.0

y1.max(axis= 1)
array([3., 6.])

y1.cumsum()
array([ 1.5,  3.5,  6.5, 10.5, 15.5, 21.5])

```

```

y1.cumsum(axis= 1)
array([[ 1.5,  3.5,  6.5],
       [ 4. ,  9. , 15. ]])

x.mean()
2.0

x.std()
0.816496580927726

np.median(x)
2.0

np.corrcoef(x)
1.0

```

Task 1.8: Copying and Sorting Arrays

- **Description:** Created a view and copies of an array and sorted 1-D and 2-D arrays in place, with and without the axis parameter.
- **Code & Output:**

```

h1= x.view()
h1
array([1, 2, 3])

h2= np.copy(x)
h2
array([1, 2, 3])

np.copy(x)
array([1, 2, 3])

x.sort()
x
array([1, 2, 3])

x1= np.array([(1, 2, 1, 4), (2, 4, 1, 10)])
x1.sort()
x1
array([[ 1,  1,  2,  4],
       [ 1,  2,  4, 10]])

x1.sort(axis= 1)

```

```
x1
array([[ 1,  1,  2,  4],
       [ 1,  2,  4, 10]])

x1.sort(axis= 0)
x1
array([[ 1,  1,  2,  4],
       [ 1,  2,  4, 10]])
```

Task 1.9: Inspecting Arrays and Type Conversion

- **Description:** Inspected an array's shape, length, number of dimensions, and data type, and converted it between integer and float with astype.
- **Code & Output:**

```
print(f"X shape: {x.shape}")
print(f"X len: {len(x)}")
print(f"X ndim: {x.ndim}")
print(f"X dtype: {x.dtype}")
print(f"X dtype name: {x.dtype.name}")
print(f"X Type conversion (int): {x.astype(int)}")
print(f"X Type conversion (float): {x.astype(float)}")
X shape: (3,)
X len: 3
X ndim: 1
X dtype: int64
X dtype name: int64
X Type conversion (int): [1 2 3]
X Type conversion (float): [1. 2. 3.]
```

Task 1.10: Indexing, Slicing, and Boolean Filtering

- **Description:** Selected elements by index, slice, reversed slice, boolean condition, and lists of row and column indices on 1-D and 2-D arrays.
- **Code & Output:**

```
print(x[1])
2

y1[0, 2]
3.0

x[0: 2]
array([1, 2])
```

```

y1[0:2, 1]
array([2., 5.])

y1[0:2, 0]
array([1.5, 4. ])

y1[:1]
array([[1.5, 2. , 3. ]])

y1[:2]
array([[1.5, 2. , 3. ],
       [4. , 5. , 6. ]])

y1[0:2]
array([[1.5, 2. , 3. ],
       [4. , 5. , 6. ]])

y1[1:2]
array([[4., 5., 6.]])

y1[:, :-1]
array([[4. , 5. , 6. ],
       [1.5, 2. , 3. ]])

y1[y1 < 4]
array([1.5, 2. , 3. ])

b= np.array([(1, 2, 3, 4), (5, 6, 7, 8), (9, 10, 11, 12), (13, 14, 15, 16)])
b.shape
(4, 4)

b[[0, 2], [1, 3]]
array([ 2, 12])

b[[0, 1, 2]][:]
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])

b[[0, 1, 2]][:, [0, 2, 3]]
array([[ 1,  3,  4],
       [ 5,  7,  8],
       [ 9, 11, 12]])

```

Task 1.11: Transposing and Reshaping Arrays

- **Description:** Transposed a 4x4 array with `.T` and `np.transpose`, flattened it with `ravel`, and changed its shape with `reshape` and the in-place `resize`.
- **Code & Output:**

```
bT= b.T
bT
array([[ 1,  5,  9, 13],
       [ 2,  6, 10, 14],
       [ 3,  7, 11, 15],
       [ 4,  8, 12, 16]])

b1T= np.transpose(b)
b1T
array([[ 1,  5,  9, 13],
       [ 2,  6, 10, 14],
       [ 3,  7, 11, 15],
       [ 4,  8, 12, 16]])

b.ravel()
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16])

b.reshape((2, -2))
array([[ 1,  2,  3,  4,  5,  6,  7,  8],
       [ 9, 10, 11, 12, 13, 14, 15, 16]])

b.reshape(4, -2)
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12],
       [13, 14, 15, 16]])

b.resize((8, 2))
b
array([[ 1,  2],
       [ 3,  4],
       [ 5,  6],
       [ 7,  8],
       [ 9, 10],
       [11, 12],
       [13, 14],
       [15, 16]])
```

Task 1.12: Adding, Removing, Combining, and Splitting Arrays

- **Description:** Added, inserted, and deleted elements with `np.append`, `np.insert`, and `np.delete`; joined arrays with `concatenate`, `vstack`, `hstack`, `column_stack`, `np.r_`, and `np.c_`; and split an array with `hsplit` and `vsplit`. Note that `b` was resized to `(8, 2)` in the previous task, so the splits operate on that shape.
- **Code & Output:**

```
bsum= np.append(b, x)
bsum
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,  1,
        2,  3])

binsert= np.insert(b, 0, 24)
binsert
array([24,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16])

bdel= np.delete(b, [1, 7])
bdel
array([ 1,  3,  4,  5,  6,  7,  9, 10, 11, 12, 13, 14, 15, 16])

print(f"Y1: {y1}\nShape: {y1.shape}\n")
print(f"Y2: {y2}\nShape: {y2.shape}\n")

bcon= np.concatenate((y1, y2), axis= 1)
bcon
Y1: [[1.5 2.  3. ]
      [4.  5.  6. ]]
Shape: (2, 3)

Y2: [[1 2 3]
      [4 5 6]]
Shape: (2, 3)
array([[1.5, 2. , 3. , 1. , 2. , 3. ],
       [4. , 5. , 6. , 4. , 5. , 6. ]])

print(f"Y1: {y1}\nShape: {y1.shape}\n")
print(f"Y2: {y2}\nShape: {y2.shape}\n")

bcon= np.concatenate((y1, y2), axis= 0)
bcon
Y1: [[1.5 2.  3. ]
      [4.  5.  6. ]]
Shape: (2, 3)

Y2: [[1 2 3]
```

```

[4 5 6]]
Shape: (2, 3)
array([[1.5, 2. , 3. ],
       [4. , 5. , 6. ],
       [1. , 2. , 3. ],
       [4. , 5. , 6. ]])

np.vstack((y1, y2))
array([[1.5, 2. , 3. ],
       [4. , 5. , 6. ],
       [1. , 2. , 3. ],
       [4. , 5. , 6. ]])

np.r_[y1, y2]
array([[1.5, 2. , 3. ],
       [4. , 5. , 6. ],
       [1. , 2. , 3. ],
       [4. , 5. , 6. ]])

np.hstack((y1, y2))
array([[1.5, 2. , 3. , 1. , 2. , 3. ],
       [4. , 5. , 6. , 4. , 5. , 6. ]])

print(f"x1: {x1}\nShape: {x1.shape}\n")
print(f"Y2: {y2}\nShape: {y2.shape}\n")

np.column_stack((x1, y2))
x1: [[ 1  1  2  4]
     [ 1  2  4 10]]
Shape: (2, 4)

Y2: [[1 2 3]
     [4 5 6]]
Shape: (2, 3)
array([[ 1,  1,  2,  4,  1,  2,  3],
       [ 1,  2,  4, 10,  4,  5,  6]])

np.c_[x1, y1]
array([[ 1. ,  1. ,  2. ,  4. ,  1.5,  2. ,  3. ],
       [ 1. ,  2. ,  4. , 10. ,  4. ,  5. ,  6. ]])

np.hsplit(b, 2)
[array([ 1],
       [ 3],
       [ 5],
       [ 7],
       [ 9],

```

```

        [11],
        [13],
        [15]]),
array([[ 2],
       [ 4],
       [ 6],
       [ 8],
       [10],
       [12],
       [14],
       [16]])]

np.vsplit(b, 2)
[array([[1, 2],
       [3, 4],
       [5, 6],
       [7, 8]]),
array([[ 9, 10],
       [11, 12],
       [13, 14],
       [15, 16]])]

```

Task 2: Loading and Visualizing Images

- **Description:** Loaded images into NumPy arrays with OpenCV and PIL, displayed them with Matplotlib, applied a Gaussian blur, and saved the results, paying attention to the BGR and RGB channel orders.

Task 2.1: Loading Images with OpenCV and Converting BGR to RGB

- **Description:** Loaded two images into NumPy matrices with cv2.imread, converted them from BGR to RGB with cv2.cvtColor so Matplotlib shows the correct colors, and displayed them side by side in a 1x2 subplot grid.
- **Code & Output:**

```

import cv2

img1= cv2.imread("./images/image.png") # Load the image data into a numpy matrix
img2= cv2.imread("./images/image copy.png") # Load the image data into a numpy matrix

img1rgb= cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
img2rgb= cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)

import matplotlib.pyplot as plt

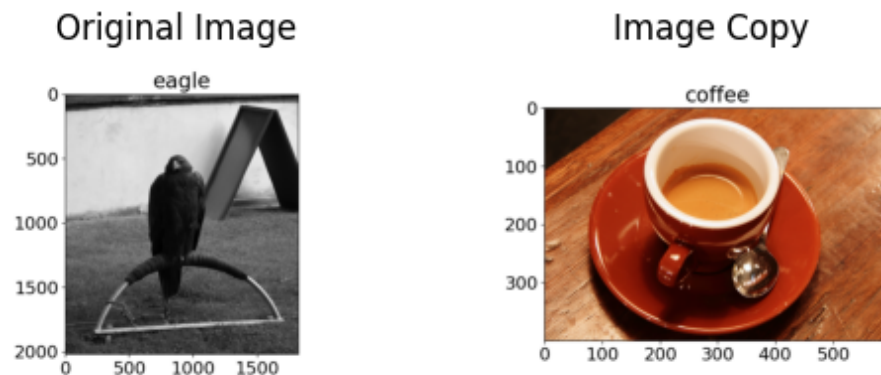
```

```
# This will create the 1x2 subplot grid
fig, ix = plt.subplots(1, 2)

# Plot the first image on the left axis
ix[0].imshow(img1rgb)
ix[0].set_title("Original Image")
ix[0].axis("off") # Hides the x and y axis ticks

# Plot the second image on the right axis
ix[1].imshow(img2rgb)
ix[1].set_title("Image Copy")
ix[1].axis("off")

# Display the figure
plt.show()
```



Task 2.2: Loading Images with PIL

- **Description:** Opened the same two images with PIL (`Image.open`) and displayed them with Matplotlib using the `Greys_r` colormap. Because the images have three color channels, the colormap has no effect and the original colors are shown.
- **Code & Output:**

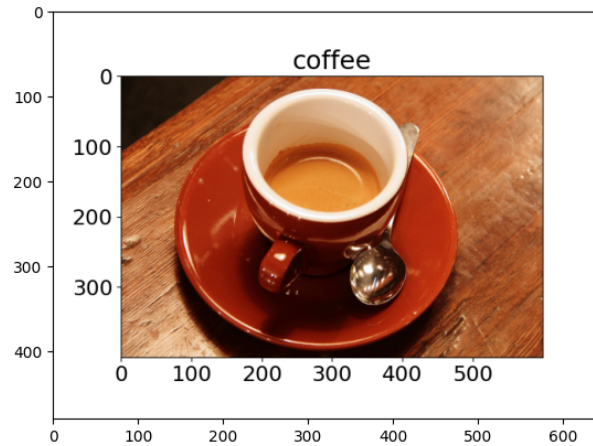
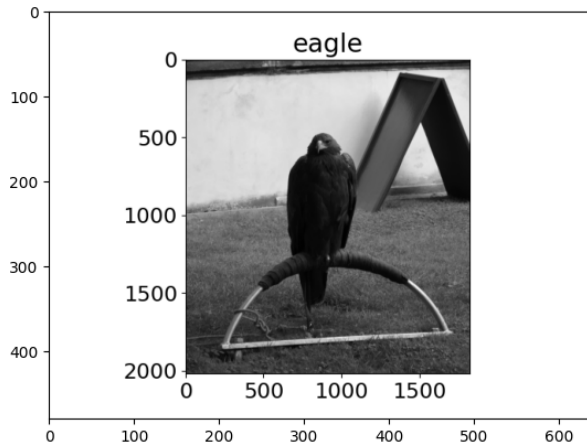
```
from PIL import Image

img1pil= Image.open("./images/image.png")
img2pil= Image.open("./images/image copy.png")

import matplotlib.pyplot as plt
import matplotlib.cm as cm

plt.imshow(img1pil, cmap= cm.Greys_r)
plt.show()
```

```
plt.imshow(img2pil, cmap= cm.Greys_r)
plt.show()
```

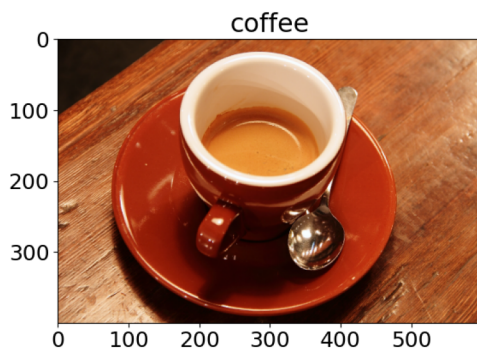


Task 2.3: Saving the RGB Image with OpenCV

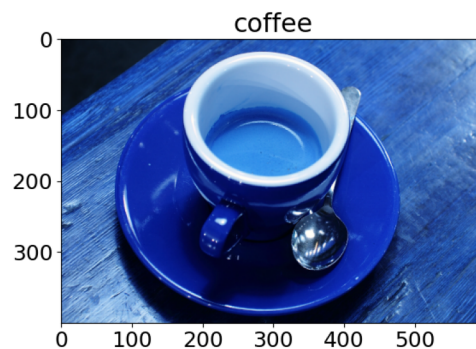
- **Description:** Saved the RGB version of the coffee image with cv2.imwrite. The call returned True, but OpenCV writes channels in BGR order, so the saved file has its red and blue channels swapped.
- **Code & Output:**

```
cv2.imwrite("./images/New1img2rgb.png", img2rgb)
True
```

image copy.png (original)



New1img2rgb.png
cv2.imwrite(img2rgb)



Task 2.4: Gaussian Blur and Saving with OpenCV and PIL

- **Description:** Converted the PIL image to a NumPy array, applied a Gaussian blur with `cv2.GaussianBlur` (`sigmaX= 3`), and displayed the result. Saved the blurred image with both `cv2.imwrite` and PIL's `save`, once from the RGB array and once from the BGR-converted array. The files whose array order does not match what the library expects (`Img2pilbblurry.png` and `Pilimg21.png`) have swapped colors, while the other two keep the original colors.
- **Code & Output:**

```
image_array= np.array(img2pil)
new_vis= cv2.GaussianBlur(image_array, ksize= (0, 0), sigmaX= 3)

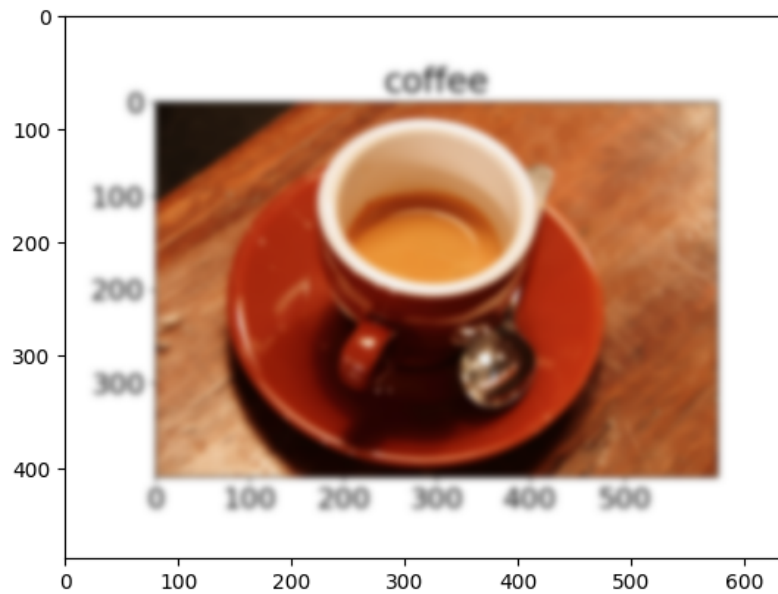
plt.imshow(new_vis)
plt.show()

image_saving= cv2.cvtColor(new_vis, cv2.COLOR_RGB2BGR)

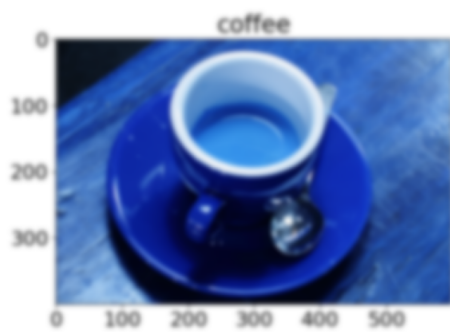
cv2.imwrite("./images/Img2pilbblurry.png", new_vis)
cv2.imwrite("./images/Img2pilbblurry2.png", image_saving)
True

image_pil1= Image.fromarray(image_saving)
image_pil2= Image.fromarray(new_vis)

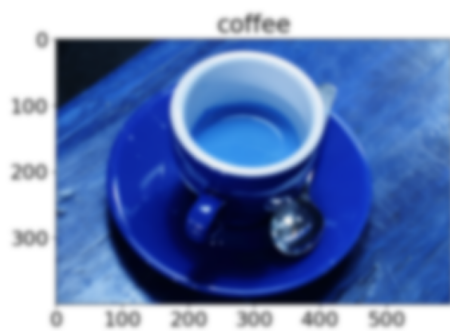
image_pil1.save("./images/Pilimg21.png")
image_pil2.save("./images/Pilimg22.png")
```



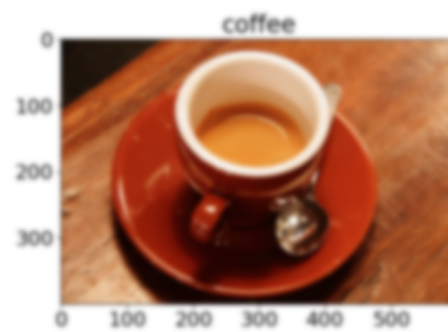
Img2pilblurry.png
cv2.imwrite(new_vis) [RGB array]



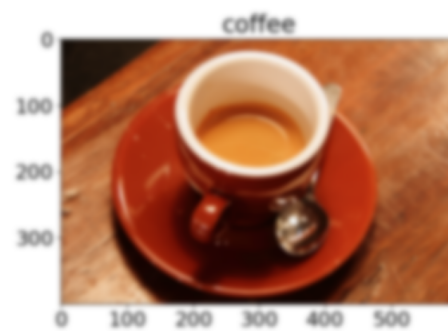
Pilimg21.png
Image.fromarray(image_saving) [BGR array]



Img2pilblurry2.png
cv2.imwrite(image_saving) [BGR array]



Pilimg22.png
Image.fromarray(new_vis) [RGB array]



Task 2.5: Inspecting Image Arrays

- **Description:** Checked the shape and pixel values of the loaded images. Both are 480x640 arrays with three channels (height, width, channels) of uint8 values, and the corner pixels are white (255, 255, 255). The array printouts below are shortened to fit the page.
- **Code & Output:**

```
img1rgb.shape
(480, 640, 3)

img1rgb
array([[255, 255, 255],
       [255, 255, 255],
       [255, 255, 255],
       ...,
       [255, 255, 255],
       [255, 255, 255],
       [255, 255, 255]],
      dtype=uint8)

pil_numpy = np.array(img2pil)
pil_numpy.shape
(480, 640, 3)

pil_numpy
array([[255, 255, 255],
       [255, 255, 255],
       [255, 255, 255],
       ...,
       [255, 255, 255],
       [255, 255, 255],
       [255, 255, 255]],
      dtype=uint8)
```

3. What I Learned

- Mathematical operations require arrays to have compatible shapes, otherwise an error is raised.
- NumPy handles diverse data types natively, including booleans, floating-point numbers, integers, and complex numbers.
- Digital images are fundamentally represented as multi-dimensional numerical arrays, which allows them to be programmatically modified and analyzed using Python.